

Troubleshooting & Tuning

Your companion to **The Guide**. Two halves:

- [Troubleshooting](#) — something broke; find your symptom and follow the exact fix.
 - [Tuning](#) — the app runs but answers are weak; three dials fix it.
-

Troubleshooting

Something not behaving? Find your symptom in the list below and follow the steps for it. These are the problems people hit most often, each with the exact fix.

A few things that apply throughout:

- **On Windows, use the WSL2 (Ubuntu) terminal** — not PowerShell. Your prompt should look like `you@PC:~$`. On **Mac**, use the built-in **Terminal**.
- **Run terminal commands from inside the `self-hosted-ai-starter-kit` folder** unless a step says otherwise. (`docker ps` and `docker exec` work from any folder.)
- Restarting or recreating a container is safe — your work lives in Docker volumes. Only `docker compose down -v` deletes your data.

Find your symptom:

1. [Read File says "Access to the file is not allowed"](#)
 2. [Read File says "No file\(s\) found"](#)
 3. [A node spins on "Executing node..." forever, with no error](#)
 4. [The embeddings model isn't in the dropdown](#)
 5. ["npm run dev" says "vite: command not found"](#)
 6. [Your web page says "I don't know" or shows no answer](#)
 7. [Your chat answers are weak or say "I don't know"](#) → see [Tuning](#)
-

1. Read File says "Access to the file is not allowed"

What's happening: n8n only reads files from one allowed folder, and the setting that points it at `/data/shared` isn't active in your running n8n. Usually it's missing from your `.env`, n8n was started before the setting was added, or (on Windows) your `.env` picked up Windows-style line endings.

Work through these in order.

First, check the setting is in your `.env`. In your terminal, run:

```
grep N8N_RESTRICT .env
```

If nothing prints, add it. In your terminal, run:

```
printf
'\nN8N_RESTRICT_FILE_ACCESS_TO=/data/shared\nN8N_BLOCK_FILE_ACCESS_TO_N8N_FILES=false\n'
>> .env
```

Don't run `cp .env.example .env` if you've already set things up — that overwrites your encryption key and database password, which breaks the credentials you created.

Now restart n8n so it picks up the setting (a setting in `.env` only takes effect when the container starts). In your terminal, run:

```
docker compose --profile cpu up -d --force-recreate n8n
```

Check the setting is now active. In your terminal, run:

```
docker exec n8n printenv N8N_RESTRICT_FILE_ACCESS_T0
```

It should print `/data/shared` .

On Windows, also check for Windows line endings. In your terminal, run:

```
docker exec n8n printenv N8N_RESTRICT_FILE_ACCESS_T0 | cat -A
```

If you see `/data/shared^M$` , that `^M` is a Windows carriage return — it sneaks in when `.env` is edited in a Windows editor. Remove it, then restart n8n. In your terminal, run:

```
sed -i 's/\r$//' .env
```

Then run:

```
docker compose --profile cpu up -d --force-recreate n8n
```

2. Read File says "No file(s) found"

What's happening: you gave n8n your computer's path. n8n runs inside Docker and only sees `/data/shared/...` .

In the Read File node's **File(s) Selector**, use the path n8n sees:

```
/data/shared/corpus/machine-learning-yearning.pdf
```

Never use `/Users/...` (Mac), `C:\...` or `/mnt/c/...` (Windows).

Check the file is actually inside the container. In your terminal, run:

```
docker exec n8n ls /data/shared/corpus/
```

If it's not listed, copy it in. From inside `self-hosted-ai-starter-kit` , run:

```
cp ../sample-docs/machine-learning-yearning.pdf shared/corpus/
```

3. A node spins on "Executing node..." forever, with no error

What's happening: the n8n container is stuck — this isn't your workflow. Reading a small file is instant, so a long spin with no progress means the container needs a restart.

To confirm: in your terminal, run:

```
docker exec n8n ls
```

If `docker ps` works but that command **hangs**, the container is stuck.

To fix it:

1. Click the whale icon in your menu bar → **Quit Docker Desktop** (Force Quit if it won't close).
2. Reopen Docker Desktop and wait until the whale icon stops moving.
3. Bring the stack back. In your terminal, run:

```
docker compose --profile cpu up -d
```

4. Reload `http://localhost:5678` in your browser and run your workflow again.

Is it actually stuck, or just slow? During ingestion the spinner is normal **as long as** your CPU is busy and the Qdrant count is climbing — a full document can take a minute or two. Stuck means no activity at all for a minute or two.

4. The embeddings model isn't in the dropdown

What's happening: in the **Embeddings Ollama** node, the Model dropdown shows only `llama3.2`, not `nomic-embed-text`. The dropdown lists only the models that are actually downloaded into Ollama — and the embeddings model hasn't been pulled yet.

See which models you have. In your terminal, run:

```
docker exec ollama ollama list
```

Pull the embeddings model. In your terminal, run:

```
docker exec ollama ollama pull nomic-embed-text
```

Wait for it to print **success**, then check both are now there. In your terminal, run:

```
docker exec ollama ollama list
```

Back in n8n, reopen the **Embeddings Ollama** node (or refresh the browser tab). `nomic-embed-text` now appears in the Model dropdown — select it.

The Embeddings node must use `nomic-embed-text`, not `llama3.2` — they do different jobs, and the embeddings model must match the one used when you ingested the document.

5. "npm run dev" says "vite: command not found"

What's happening: the page's packages aren't installed yet. A fresh copy of the repo has no `node_modules` folder, so you have to install once before you can run the page.

In the `frontend` folder, install the packages. In your terminal, run:

```
npm install
```

Then start the page. In your terminal, run:

```
npm run dev
```

It prints a local address — open `http://localhost:5173` in your browser.

If **node** or **npm** is missing (you get `command not found`), or `node -v` shows a version below 18, install Node first:

- **Mac** — in your terminal, run:

```
brew install node
```

- **Windows (WSL2)** — install Node with nvm. In your terminal, run:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

then run:

```
exec $SHELL
```

then run:

```
nvm install --lts
```

Check it worked — `node -v` should show v18 or higher — then run `npm install` and `npm run dev` again.

6. Your web page says "I don't know" or shows no answer

If the page shows *anything* back — even "I don't know" — your webhook is working. The publish step, CORS, and the URL are all fine. The issue is the answer itself. There are two separate causes.

6a. The question isn't reaching the workflow.

Test the webhook directly. In your terminal, run:

```
curl -s -X POST http://localhost:5678/webhook/rag-chat -H 'Content-Type: application/json' -d '{"question":"What is the difference between bias and variance?"}'
```

If the answer says something like *"no question provided"*, the question isn't wired into the chain. In the **RAG webhook** workflow, open the **Question and Answer Chain** node and check:

- **Source for Prompt (User Message)** is set to **Define below**.
- **Prompt (User Message)** is in **Expression** mode and reads `{{ $json.body.question }}`.
- The grey preview under that field shows your actual question, not a blank.

6b. The question gets through, but the answer is weak ("I don't know").

The **RAG webhook** workflow has its **own** nodes — tuning your chat workflow doesn't change them. Apply the same three settings here (see [Tuning](#) below): set the Retriever **Limit** to 8, set the Ollama Chat Model's **Context Length** to 4096, and replace the chain's default **System Prompt Template**.

Also check: the workflow is **Active** (published), and the Webhook node's **Allowed Origins (CORS)** is set to `*`. If the workflow isn't Active, the page gets "Failed to fetch".

7. Your chat answers are weak or say "I don't know"

Small models hold back, and the chain's default prompt actually tells them to. This isn't a bug — it's tuning. See the [Tuning](#) section below for the three dials that fix it.

Tuning: getting better answers

If the chat keeps replying "**I don't know**" even though the document covers the question, it's almost always one of three dials. The default setup is deliberately *conservative* (good against hallucination, bad for confident answers), so you have to loosen it.

*Do all three below, then ask your question again. Order of impact: **Limit** → **prompt** → **num_ctx**.*

Before you tune — three quick sanity checks

These *also* cause "I don't know," and no amount of tuning fixes them:

1. **Is the question actually answerable from the document?** If it isn't in the source, "I don't know" is the *correct* answer.
2. **Did ingestion populate Qdrant?** The collection must have vectors (you should see a few hundred). In your terminal, run:

```
curl -s http://localhost:6333/collections/documents | grep -o '"points_count": [0-9]*'
```

If it's `0`, re-run the ingestion workflow first.

3. **Same embedding model on both sides?** The retriever's **Embeddings Ollama** must be `omic-embed-text` — the *same* model used at ingestion — or the query vector won't match the stored vectors and retrieval returns nothing useful.

The three dials (in the "Chat with docs" workflow)

1. **Retriever Limit** → **8**. More retrieved chunks = more context for the model, so a small model stops hedging. Click the **Vector Store Retriever** node → set **Limit** to `8` (default is `4`). *Don't over-raise it — `6` — `8` is the sweet spot for a document this size; too high dilutes the answer and slows it down.*

2. **Ollama Chat Model context window (num_ctx)** → **4096**. Easy to miss. If the retrieved chunks plus your question exceed the model's context window, Ollama **truncates the prompt from the start** — often dropping your actual question — and you get "I don't know." Click the **Ollama Chat Model** node → **Options** → **Add Option** → **Context Length** → set `4096` (or `8192`). *(While you're there, set **Sampling Temperature** to about `0.2` for focused, factual answers. If your n8n version doesn't list "Context Length," skip it — dials 1 and 3 still help a lot.)*

3. **Replace the Q&A Chain system prompt**. The **Question and Answer Chain** ships with a default that says "...if you don't know the answer, just say that you don't know" — a big reason for the refusals. Click the **Question and Answer Chain** node → find the prompt field (the "You are an assistant..." text) → replace it with the prompt below. **Keep Context: {context} exactly** — that placeholder is where n8n injects the retrieved chunks.

You are a precise assistant answering questions about the provided document.
Use **ONLY** the retrieved context below to answer. Be direct and include as much relevant detail as the context supports. If the context partially answers the

```
question, give that partial answer rather than refusing. Only say you don't know
if the context contains no relevant information at all. Do not invent facts that
are not in the context.
```

```
-----
```

```
Context: {context}
```

This removes the reflexive "just say you don't know" and tells the model to give a partial answer rather than refuse, but **keeps it grounded** ("use ONLY the context", "don't invent facts") so you don't trade refusals for hallucinations. **Save the workflow → ask your question again.**



The webhook workflow is separate. The **RAG webhook** has its own copy of these nodes — tuning "Chat with docs" doesn't touch it. Apply the same three dials there too (see symptom 6b above).

Optional: a bigger model

A larger model follows instructions more confidently. In your terminal, run:

```
docker exec ollama ollama pull mistral
```

Then in the **Ollama Chat Model** node, set **Model** to `mistral` (7B) instead of `llama3.2` (3B) and compare.